



"I've got it, too, Omar ... a strange feeling
like we've just been going in circles."

Reuso: Framework II (blackbox)

Federico Balaguer

Resumen de la clase anterior

Frameworks:

- Proveen una solución reusable para una familia de aplicaciones
- Las clases en el framework se relacionan (herencia, conocimiento, envío de mensajes) de manera que resuelven la mayor parte del problema en cuestión
- El código del framework controla/usa al código de la instancia
- Tipos de Frameworks
 - Aplicación: desktop, webapps, tcpserver :-)
 - Manejo Datos: ORDB, pipelines, NRDB
 - Sistemas Distribuidos: mensajes, eventos, rpc
 - Testing: unit, web pages
- Framework de Caja Blanca: las instancias “completan” el hilo de control agregando código.
 - Ejercitando un hotspot con herencia
 - Modificando código fuente del framework

Dominio: TCP Servers

SingleThreadTCPServer (whitebox)

Cookbook:

1. Crear Subclase de SimpleThreadTCPServer
 - a. Debe implementar Main(String[])
 - i. crear una instancia
 - ii. enviar método startLoop(String[])
 - b. Debe implementar handleMessage(String) ⇒ hook

tcp.server.reply (blackbox)

Cookbook

1. En un objeto “contexto”
 - a. instanciar un MessageHandler
 - i. Echo, Hash, Passwd, RailFence, etc
 - b. Instanciar ConnectionHandler requiere un MessageHandler
 - i. SimpleConnectionHandler
 - ii. MultiConnectionHandler ¹
 - c. Instanciar TCPControlLoop requiere un ConnectionHandler
 - d. Enviar método startLoop() al TCPControlLoop

Insntanciacion ⇒

TCPControlLoop(ConnectionHandler(MessageHandler))

1 Multiples clientes conectados simultaneamente

```

1  import java.io.PrintWriter;
2
3  public class EchoServer extends SingleThreadTCPServer {
4
5      public void handleMessage(String message, PrintWriter out) {
6          out.println(message);
7      }
8
9      Run | Debug
10     public static void main(String[] args) {
11
12         new EchoServer().startLoop(args);
13     }
14 }

```

SingleThreadTCPServer
(hotspot herencia)

```

1  import tcp.server.reply.*;
2
3  public class EchoApp {
4
5      Run | Debug
6      public static void main(String[] args) {
7
8          new TCPControlLoop(new SingleConnectionHandler(new EchoHandler())).startLoop(args);
9      }
10 }

```

tcp.server.reply
(hotspot composición)

```
1 import tcp.server.reply.*;
2
3 public class MultiEchoApp {
4
5     Run | Debug
6     public static void main(String[] args) {
7
8         new TCPControlLoop(new MultiConnectionHandler(new EchoHandler())).startLoop(args);
9     }
10 }
```

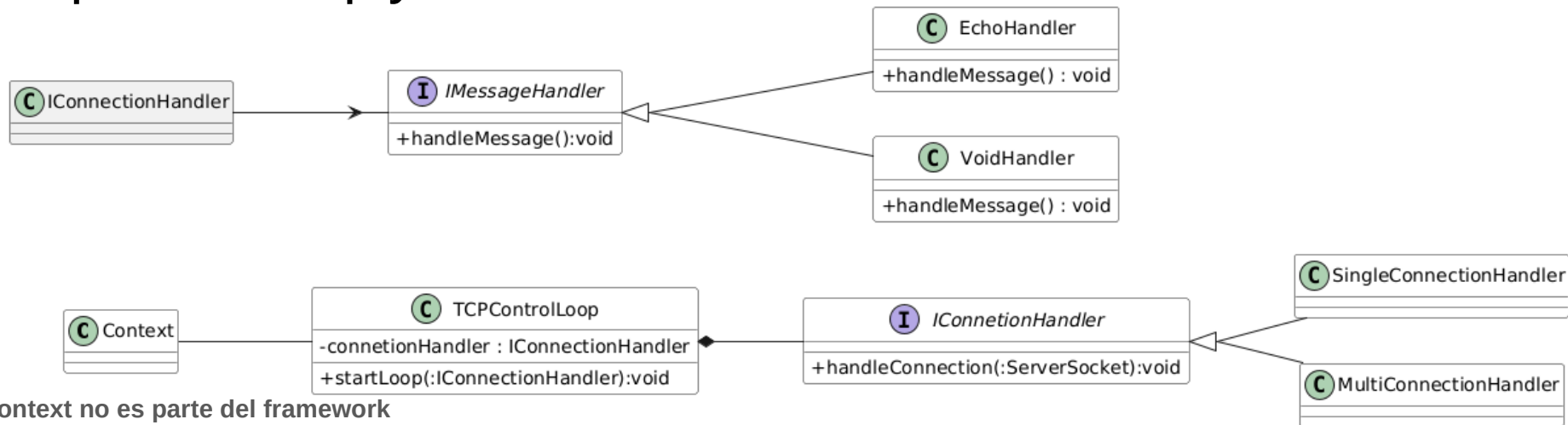
tcp.server.reply
multisession, echo handler

```
1 import tcp.server.reply.*;
2
3 public class TestApp {
4
5     Run | Debug
6     public static void main(String[] args) {
7
8         new TCPControlLoop().startLoop(args);
9     }
10 }
```

tcp.server.reply
singlesession, void handler

Dependency injection

tcp.server.reply



FrozenSpot:

Condición de corte de la conexión

Un tipo de conexión (runtime)

Un tipo de MessageHandler (runtime)

HotSpot:

Nuevos MessageHandlers s/funcionalidad

Hash, timestamp, etc

Nuevos ConnectionHandlers

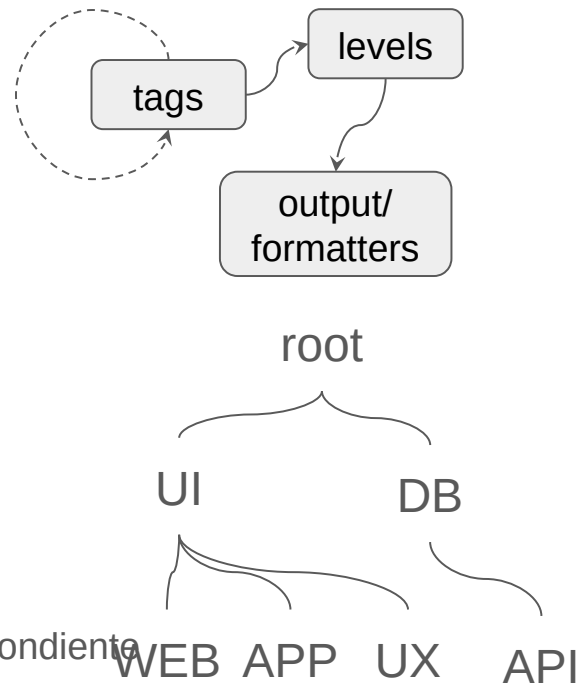
Timeout, recording, etc

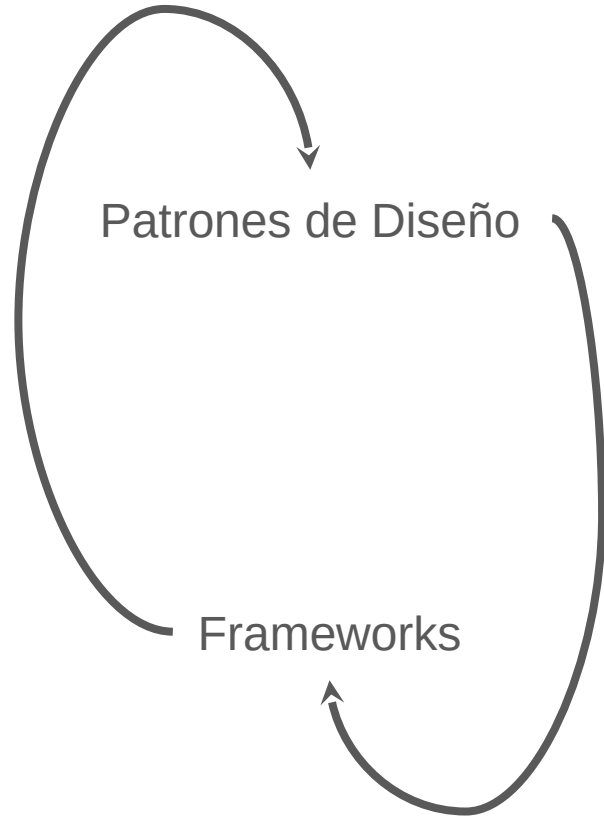
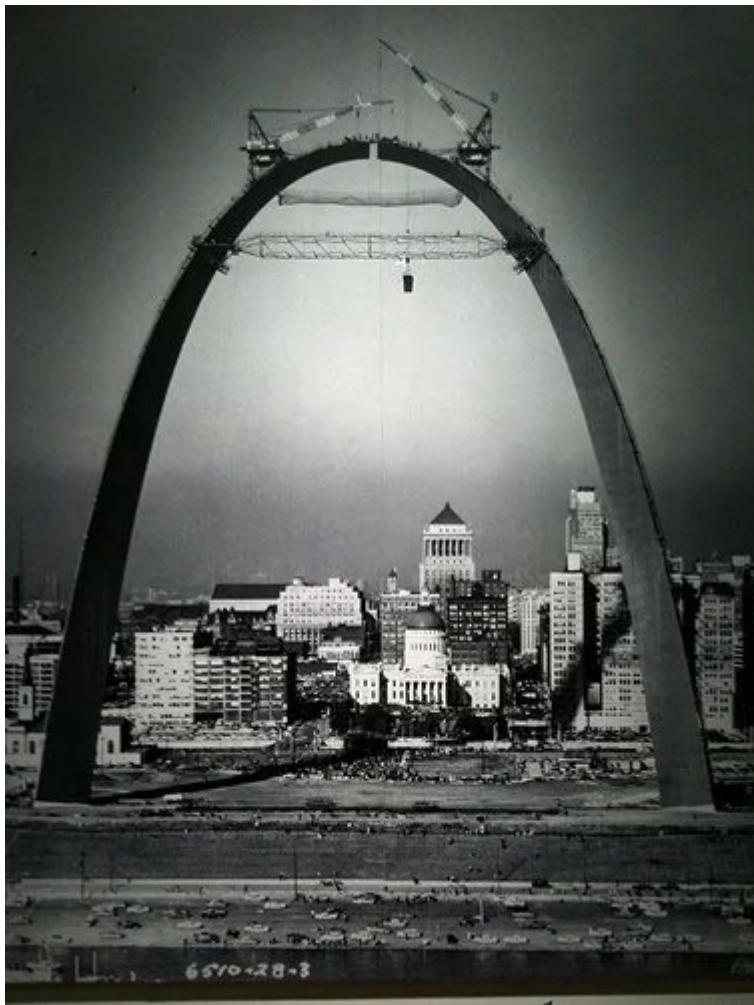
tcp.server.reply: wrap up

- TCPControlLoop se configura con TCPConnection y MessageHandler
- El contexto puede ser:
 - servidor
 - parte de una aplicación
- Posibles mejoras:
 1. Crear una Superclase de SingleConnectionHandler y MultiConnectionHandler
 2. Crear la jerarquía de EndSessionPolicy
 3. Modelar el concepto de Session
 - a. En SingleConnectionHandler la sesión es el loop que procesa mensajes
 - b. En MultiConnectionHandler la sesión es el TCPWorker (subclase de Thread)

Java.util.logging (in a nutshell)

- Define diferentes “tag” (== logger)
 - Estructura jerárquica (árbol) de tags
- Permite implementar
 - Esquemas simples de logging
 - Esquemas complejos de logging
 - Extensiones para necesidades particulares
- Los desarrolladores
 - Realizan la configuración que van a utilizar
 - Tags, levels, output/formatter
 - Invocan mensajes de logging considerando el logger correspondiente
- Presentación en moodle
 - Pdf + audio
 - <https://docs.oracle.com/en/java/javase/11/core/java-logging-overview.html>
 - <https://docs.oracle.com/en/java/javase/11/docs/api/java.logging/java/util/logging/package-summary.html>





Diseño de Frameworks

- Frameworks están diseñados para reuso
 - Instanciación
 - Extensión
- Un Patrón de Diseño es: s__n a p__a r____e
 - Consecuencia positiva relacionada con un tipo de cambio
- Paradoja del Diseño de Frameworks
 - Frameworks deben ser reusables
 - El reuso ocurre cuando el Framework es utilizado
 - ⇒ Un Framework no puede ser reusable por diseño (antes de ser usado)
- El reuso de un framework emerge
 - Como la arquitectura en un proyecto desarrollado con métodos ágiles
 - “Un framework emerge de resolver al menos 3 ejemplos” — Ralph Johnson

⇒ La evolución en el diseño suele estar asociado con patrones de diseño

...en SimpleThreadTCPServer

HotSpots vs FrozenSpot

FrozenSpot: aspecto del framework que afecta a todas las instancias y que no se puede modificar (marca indeleble)

Ej: SingleThreadTCPServer cierra la conexión con el cliente si:

1. El cliente cierra la conexión
2. El cliente envía un mensaje vacío ¹

—Todas las aplicaciones de este framework tienen esta “marca”/arquitectura

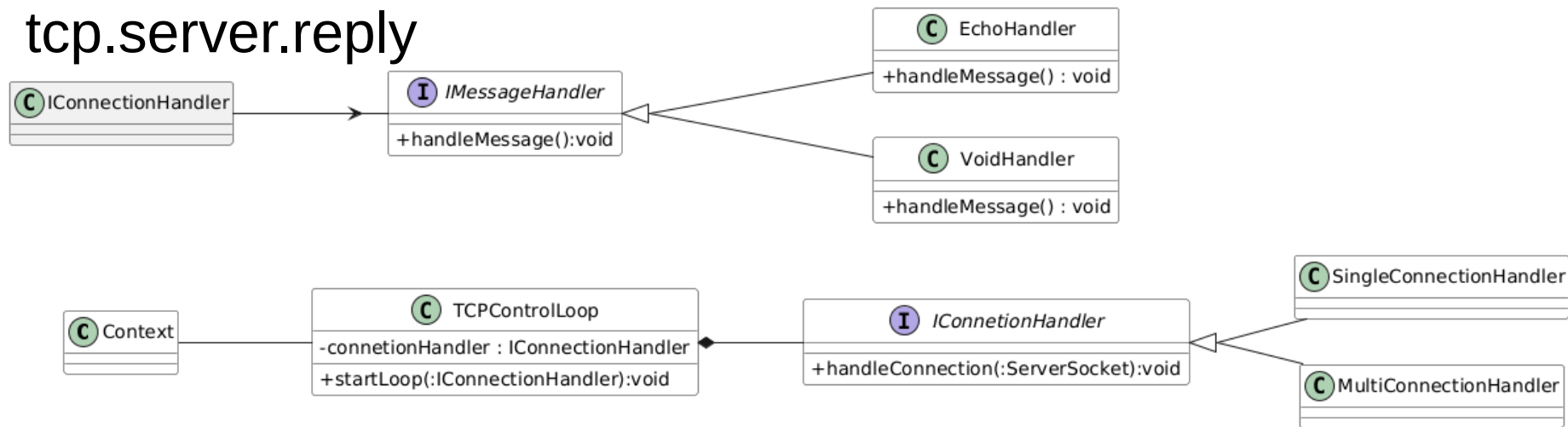
```
72 |         if (inputLine.equalsIgnoreCase("")) {  
73 |             break; // Client requested to close the connection  
74 |         }
```

Ejercicio/Evaluación: como podría parametrizar la “condición de cierre de sesión” ?

(1) SingleThreadTCPServer.java: 72-74

- Cómo parametrizar el final de la conexión?
 - Variable instancia para evitar hardcoding el string vacío
 - Jerarquía de “ConnectionKillerPolicy” (es Strategy?)
 - Inversion de Control:
 - Mover condición de corte a un método
 - SimpleThreadTCPServer delega en la subclase que instancia el framework.

tcp.server.reply



- SingleConnectionHandler y MultiConnectionHandler implementa esto:

```
if (inputLine.equalsIgnoreCase("")) {
    break; // Client requested to close the connection
}
```

- Cómo parametrizar el final de la conexión?
 - No hay subclase para delegar
 - Necesario considerar: message count, time-out, token-based, etc

Resumiendo I

- Frameworks presentan una manera de reuso que supera el reuso (solo de código)
- Un Framework dicta la manera de ejecutar un programa (execution thread)
- El diseño de un framework foco en una manera de describir un dominio o en las cosas importantes de un dominio
- Frameworks vistos en OO2
 - SingleThreadTCPServer: Un servidor como extensión de una clase loop + sesión (singular)
 - Tcp.server.reply: loop(sesión(msgHandler)). Sesión singular o multiple. Diferentes MsgHandler
 - Java.util.logging: jerarquía de Labels + composición de filtros, formatos y salidas
- Diseños de Frameworks
 - FrozenSpots: partes del diseño que no cambian
 - HotSpots: elementos del diseño pensadas para adaptarse
 - Hook Methods
 - Patrones de Diseño (Clases/Jerarquías, Delegación, Protocolos)
 - El diseño se va adaptando a los problemas comunes en un dominio
 - Problemas comunes ⇒ design patterns
 - En CajaBlanca. Loop de control suele ser Template Method.
 - En tcp.server.reply MessageHandler puede ser Strategy o Command (GoF)
 - Si en el diseño de un framework identificamos un patrón, la estructura correspondiente seguramente es un hotspot.

Resumiendo II

- Tipos de Instanciación
 - Caja Blanca: las instanciaciones modifican o extienden el código fuente (loop de control + hook clases)
 - Caja Negra: las instanciaciones se basan en configuraciones

Material en Moodle

- Código fuente de tcp.server.reply
 - Framework
 - Ejemplos de instanciación
- Evolving Frameworks: A Pattern Language for Developing Object-Oriented Frameworks - D. Roberts, R. Johnson
- Hot-Spot-Driven Framework Development - W. Pree

